

Homework 2

Student Name: Lyon Kee

1. Maximum likelihood estimation for statistical language models with proper smoothing (50pts)

- 1.1. Paste your implementation of the linear interpolation smoothing and absolute discount smoothing.

```
def li_smoothing(prev_word, w, prev_w_count, TTF_sum):
    return (1-li_lambda)*bigram_count[prev_word]
        .get(w, 0)/prev_w_count + li_lambda*(TTF[w]/TTF_sum)

def ad_smoothing(prev_w, w, num_uniq_bigrams_given_word, total_bigram_count_giv
    return (max(bigram_count[prev_w].get(w, 0) - ad_delta, 0) +
        ad_delta * num_uniq_bigrams_given_word * unigram_prob[w]) /
        total_bigram_count_given_word

def bigram_smoothing(prev_word):
    prev_w_count = TTF[prev_word]
    TTF_sum = sum(TTF.values())
    num_uniq_bigrams_given_word = len(bigram_count[prev_word])
    total_bigram_count_given_word = sum(bigram_count[prev_word].values())

    for word in unigram_prob:
        if li_smooted_bigram.get(prev_word, None) is None:
            li_smooted_bigram[prev_word] = {}
            ad_smooted_bigram[prev_word] = {}
        li_smooted_bigram[prev_word][word] = li_smoothing(prev_word,
            word, prev_w_count, TTF_sum)
        ad_smooted_bigram[prev_word][word] = ad_smoothing(prev_word,
            word, num_uniq_bigrams_given_word, total_bigram_count_given_word)

def sort_bigram(prev_word):
    li_smooted_bigram[prev_word] = {key: val
        for key, val in
            sorted(li_smooted_bigram[prev_word].items(),
                key=lambda pair:pair[1], reverse=True)}
    ad_smooted_bigram[prev_word] = {key: val
        for key, val in
            sorted(ad_smooted_bigram[prev_word].items(),
                key=lambda pair:pair[1], reverse=True)}
```

```

li_lambda = 0.9
ad_delta = 0.1
li_smooted_bigram = {}
ad_smooted_bigram = {}

# Perform smoothing on bigram
bigram_smoothing('good')

print("sanity check li_smooted_bigram: ", sum(li_smooted_bigram['good'].values()))
print("sanity check ad_smooted_bigram: ", sum(ad_smooted_bigram['good'].values()))

sort_bigram('good')

```

- 1.2. The top 10 words selected from the corresponding two bigram language models.
linear interpolation: ['the', 'and', 'i', 'a', 'to', 'but', 'it', 'wa', 'of', 'for']
absolute discount: ['but', 'and', 'the', 'i', 'as', 'food', 'it', 'thing', 'for', 'too']
- 1.3. Your explanation of the observations about the top words under those two bigram language models. Absolute discounting takes a portion of the observed words and apply it to unseen words, whereas in linear interpolation, we observe that it takes a portion from bigram and a portion from the unigram, therefore the words that differ between absolute discounting and linear interpolation is most likely due to how words are spread out and what percentage of words of bigram vs unigram we take when calculating the smoothed values given by λ and δ .
2. Generate text documents from a language model (40pts)

- 1.1. Paste your implementation of the sampling procedure from a language model.

```

import numpy as np

document_length = 20
number_of_documents = 10

unigram_documents = [[word] for word in np.random.choice(list(unigram_prob.keys()))]
li_documents = [[word] for word in np.random.choice(list(unigram_prob.keys()))]
ad_documents = [[word] for word in np.random.choice(list(unigram_prob.keys()))]

print("unigram_documents: ", unigram_documents)
print("li_documents: ", li_documents)
print("ad_documents: ", ad_documents)

for _ in range(document_length - 1):
    for i in range(number_of_documents):
        unigram_documents[i].append(np.random.choice(list(unigram_prob.keys()))),

```

```

# Generate word for Linear Interpolation smoothing
prev_word = li_documents[i][-1]

# Smoothing all bigrams given prev_word
prev_w_count = TTF[prev_word]
TTF_sum = sum(TTF.values())
for word in unigram_prob:
    if li_smooted_bigram.get(prev_word, None) is None:
        li_smooted_bigram[prev_word] = {}
    li_smooted_bigram[prev_word][word] = li_smoothing(prev_word, word,
# normalize prob
li_smoothed_sum = sum(li_smooted_bigram[prev_word].values())
for word in li_smooted_bigram[prev_word]:
    li_smooted_bigram[prev_word][word] /= li_smoothed_sum
li_documents[i].append(np.random.choice(list(li_smooted_bigram[prev_wor

# Generate word for Absolute Discount smoothing
prev_word = ad_documents[i][-1]

# Smoothing all bigrams given prev_word
num_uniq_bigrams_given_word = len(bigram_count[prev_word])
total_bigram_count_given_word = sum(bigram_count[prev_word].values())
for word in unigram_prob:
    if ad_smooted_bigram.get(prev_word, None) is None:
        ad_smooted_bigram[prev_word] = {}
    ad_smooted_bigram[prev_word][word] = ad_smoothing(prev_word, word,
# normalize prob
ad_smoothed_sum = sum(ad_smooted_bigram[prev_word].values())
for word in ad_smooted_bigram[prev_word]:
    ad_smooted_bigram[prev_word][word] /= ad_smoothed_sum
ad_documents[i].append(np.random.choice(list(ad_smooted_bigram[prev_wor

print("unigram_documents: ", [" ".join(word) for word in unigram_documents])
print("li_documents: ", [" ".join(word) for word in li_documents])
print("ad_documents: ", [" ".join(word) for word in ad_documents])

```

- 1.2. The 10 documents generated from $p(w)$, $p^L(w_i|w_{i-1})$ and $p^A(w_i|w_{i-1})$ accordingly, and the corresponding likelihood given by the used language model.

Unigram Documents:

01: as dont but block from go the good thick well definit farm to heart so am

veri meal would to
 probability: 2.1805299954802233e-59
 02: we is way the a fact in though foam in loud we it the food had the okay fairli
 probability: 7.408406314418808e-55
 03: i view not more to is wa a and you yeah a flan it put thi sex minut to at
 probability: 8.095873968537276e-56
 04: off in avail jacketeven the tequila pork limit we ideal a at and to bona and
 friend wari it end
 probability: 8.460164102534893e-68
 05: crab urchin chicken to upon with door de hous deal clear fri husband view to
 back that i tlacoya more
 probability: 4.1389872142428336e-69
 06: it spice to to the finish and already veri a i them fusili starsim to there howev
 price a us
 probability: 6.768186099077475e-60
 07: oyster they servic thought thi us post a sizabl crust we and id the chines lula
 next and or the
 probability: 1.0431249647962343e-61
 08: extra grill as scrambl there your is benedict best bar amazingi transact much
 had scene even and select on
 probability: 1.69418754355521e-65
 09: waitress for about menu own an or jazz to thi for bag guard size faux a at
 night which
 probability: 9.525859574458547e-63
 10: tasti achatz unmatchedth access on some good fine sure had fresh but whenev
 ring is classroom proper in lech copper
 probability: 1.4800874590249966e-79

linear Interpolation Documents:

01: hand lobster is kobe go there long chef one veri simpli ventur that stick
 factoryrestaur thing ambient im the fed
 probability: 2.739116121950086e-70
 02: i is great back for thre theyr the the the bacon rancid zeez you happi nice
 friend vegetarian lowli forget
 probability: 1.9614947289242573e-68
 03: of is food sooo good ive had were how their ba and thousand seen what we
 here tapa farm new
 probability: 1.0685499748284916e-62
 04: it by time i sit with happi do therefor all the cook flare live the qualiti
 simmer the vegan
 probability: 1.739315061291922e-63
 05: the servic food by view i to vegetarian fingerl had chicago or chees case live
 counter menu but servic nice
 probability: 3.7722215606251135e-63

06: a the we for a would with had friday is and snap those tri tabl at warm cough
place it
probability: 3.582548396506098e-57

07: parker wa pretti is her on effort star salad is each see dont and a a that
burger least highli
probability: 6.669392234583096e-58

08: first off the foraft great fishi be recommend the quesadilla and thi snatch
until a profession and im nice be
probability: 1.6097820125876702e-63

09: is as wait thing garrish in place plainli to two pretenti good further it you
lobster noodl seat i place
probability: 2.2931757778212915e-66

10: and for the their place we not busi camel finish the which neverend restaur
night ive made much night when
probability: 2.1335580733014384e-61

Absolute Discount Documents:

01: seen ill probabl ever be so littl tough and plate of a foot in to be super chees
the best
probability: 2.094829434704819e-49

02: went with just saw thi place the whole fishtri duper delish bread which is to
come in the tast great
probability: 1.190914902148415e-47

03: for a cheap date which were scrape off time here dont want to us if you cant
begin to tri
probability: 8.349002697629697e-43

04: big sandwich but iberico thoughw know if i becam a celebr it not agre with
pay for five star thi
probability: 4.381736138533007e-52

05: hitachino shogun burger cant get you fellow runner he neither offer on the
drink across the glass of oyster clam
probability: 6.854208100208281e-49

06: near the loung door but wa satisfi they made the day nice waiter and gave
our first time ive been
probability: 5.955329301426365e-47

07: recommend thi but it said the meat i have thi is not realli cool but the other
place in the
probability: 7.287396918522367e-39

08: order at first time i figur out on draft select in such a friday night to be stand
out a
probability: 4.224845144812299e-39

09: want to meet a line wa simpli amaz textur the warm and my stomachoveral i
want to tri a great
probability: 1.9813538168332654e-41

10: day but if i serious i didnt have to stay away one browni point besid you have to the place
probability: 2.0931678507605213e-41

3. Reading Assignment — Belief or Bias in Information Retrieval (10pts)

Google and Bing could implement several strategies focused on enhancing search evaluation, document retrieval, and ranking processes to address the spread of misinformation. They could prioritize credibility and accuracy over popularity such as taking popular news anchors over popular news articles. Further actions could involve analyzing the credibility of sources, cross-referencing information with reputable sources, and flagging potentially misleading content.

Both search engines could promote diversity in search results by reducing reliance on popularity metrics and instead considering a wider range of perspectives and sources by giving documents a more even page rank and evaluation on their own set of metrics. This could help mitigate the echo chamber effect where users are only exposed to information that reinforces their existing beliefs.

Furthermore, Google and Bing could invest in user education initiatives to enhance critical thinking skills and awareness of misinformation tactics. This could involve providing users with tools and resources to evaluate the credibility of information, such as browser extensions that highlight fact-checked sources or provide context about the reliability of a website, or advertisements that educate users.

Lastly, both search engines could collaborate with academic institutions and fact-checking organizations to continuously refine their algorithms and improve the accuracy of search results. By leveraging expertise from various domains, they can stay ahead of evolving misinformation tactics and provide users with more reliable information. Overall, a multi-faceted approach that combines algorithmic improvements, user education, and collaboration with external experts can help Google and Bing combat the spread of misinformation on the web.

4. Extra Credits

Classic language models, such as unigram and bigram models commonly implemented in small corpora like Yelp food reviews, rely on simple probabilistic methods to estimate the likelihood of word sequences based on their frequencies in the training data. However, trending Large Language Models (LLMs), such as GPT (Generative Pre-trained Transformer) models, represent a significant advancement over these traditional approaches. LLMs utilize deep learning techniques to capture complex relationships between words and generate coherent text based on vast amounts of pre-training data. Unlike unigram and bigram models, LLMs have the ability to understand contextual nuances, capture long-range dependencies, and generate more human-like text

responses. Additionally, LLMs can perform a wide range of natural language processing tasks, including text generation, summarization, translation, and question answering, making them highly versatile and applicable across various domains. Overall, LLMs represent a paradigm shift in language modeling, offering superior performance and capabilities compared to classic models like unigrams and bigrams.